

Kronos: A Reproducible Research Platform for Empirically Evaluating Classical Chess Search Heuristics on Managed JavaScript Runtimes

Piyush Sinha

Department of Computer Science
Kronos Research Project
Email: piyush218sinha@gmail.com

July 2026

Abstract

We present Kronos, a modular chess engine and benchmarking framework implemented in Node.js and JavaScript, designed to empirically evaluate the performance limits and execution behavior of classical search heuristics within managed, garbage-collected runtime environments. Standard chess engine implementations favor native compile targets like C/C++ to avoid execution overhead and garbage collection pauses. Kronos demonstrates that through targeted optimization of board representations, strict object-reuse patterns, and modular search configurations, stable search throughput can be achieved on dynamic runtimes. Our architecture decouples the React-based user interface control plane from background search Web Workers executing the search logic. The engine implements standard search enhancements including Principal Variation Search (PVS), Null Move Pruning (NMP), and Late Move Reductions (LMR), backed by a size-bounded, Zobrist-keyed Transposition Table. Using a parallel tournament framework with color-flipped paired matches, we show that these combined heuristics compress the search space by 62.0% at depth 4 and 83.2% at depth 8 compared to raw alpha-beta search. Calibration against depth-limited Stockfish 18 anchors establishes a peak playing strength of 1485 ± 86 Elo at search depth 6, after which search-latency timeouts degrade tactical performance. These outcomes suggest that carefully structured dynamic runtimes can support stable, real-time game-tree search without performance degradation from memory sweeps.

Executive Summary

This report presents the engineering design and empirical findings of Kronos, a modular chess engine and benchmarking framework implemented in Node.js/JavaScript. Kronos serves as a reproducible systems testbed for game-tree search heuristics in managed, garbage-collected runtimes.

V8 Engineering Solutions

Historically, dynamic runtimes were considered unsuitable for latency-sensitive search systems due to event loop freezing and garbage collection (GC) sweeps. Kronos resolves these issues through three implementations:

1. **Asynchronous Worker Decoupling:** Decoupled background Web Workers communicate with the React UI via serialized JSON messages, preventing interface freezes.
2. **GC-Neutral Search Path:** Direct in-place board state mutations bypass object allocations, and a size-bounded transposition table limits dynamic heap growth.
3. **JIT Compiler Warming:** Benchmarks run pre-heat iterations to trigger V8 JIT compilation and optimize execution paths prior to recording search telemetry.

Key Experimental Outcomes

* **Heap Bounded Memory:** Heap usage remains stable between 14.8 MB and 28.4 MB during continuous search, eliminating major GC sweeps. * **Search Reduction:** Late Move Reductions (LMR) is the dominant pruning heuristic, yielding a 66.3% node reduction at depth 4. * **Calibrated Strengths:** Stockfish 18 matchups calibrate the peak playing strength at 1485 ± 86 Elo (at depth 6), after which single-threaded latency constraints cause search timeouts. * **Statistical Validation:** All core search heuristics (PVS, LMR, and combined ordering) are statistically validated under Wald Sequential Probability Ratio Tests (SPRT).

This work establishes that managed runtime environments can execute complex game-tree searches efficiently and stably, providing a highly accessible web-deployable systems research framework.

Contents

Executive Summary	i
1 Introduction	1
2 Research Objectives and Contributions	3
2.1 Research Questions	3
2.2 Classical Techniques Adopted	3
2.3 Engineering Contributions	4
3 Related Work and Background	5
3.1 Game-Tree Search Foundations	5
3.2 Search Reduction and Move Ordering Heuristics	5
3.3 Transposition Caching and Managed Runtime Constraints	6
3.4 Comparison with Existing JavaScript Chess Engines	8
4 System Overview	9
4.1 User-Facing Application Platform	9
4.2 Asynchronous Worker Architecture	9
4.2.1 Software Subsystem Decomposition	11
4.3 Engine Design and Memory Optimization	12
4.3.1 GC-Neutral Board Mutation	12
4.3.2 Static Position Evaluation	12
4.3.3 Transposition Table and Memory Bounding	12
4.3.4 Engine Evolution Mapping	13
4.4 Benchmarking and Calibration Pipelines	13
4.5 Engine Development Progression	13
5 Experimental Methodology	15
5.1 Automated Tournament Scheduler	15
5.2 Execution Integrity and Verification	17
5.3 Statistical Heuristics and Formulas	17
5.4 Checkpointing and Memory Profiling	18
6 Experimental Results	19
6.1 Search Optimizations	20
6.1.1 Negamax Alpha-Beta Pruning	20
6.1.2 Principal Variation Search (PVS)	20
6.2 Move Ordering Heuristics	20

6.2.1	MVV-LVA Capture Ordering	20
6.2.2	Killer Moves and History Heuristic	20
6.3	Search Extensions and Reductions	20
6.3.1	Late Move Reduction (LMR)	20
6.3.2	Null Move Pruning (NMP)	21
6.3.3	Quiescence Search	21
6.3.4	Evaluation Function: Piece-Square Tables	22
6.3.5	Memory Engineering and GC Stability	22
6.4	Search Depth Scaling and NPS Throughput	22
6.5	Stockfish Reference Calibration	23
6.5.1	Statistical Significance of Heuristic Gains	23
6.6	Game-Level Computational Overhead	24
7	Discussion	29
7.1	GC-Neutral Memory Engineering	29
7.2	Synergy between Move Ordering and Search Reductions	29
7.3	Latency Constraints and the Search Ceiling	30
7.4	Deterministic Seeding and SPRT Validation	30
7.5	Draw-Rate Analysis in Self-Play Pools	30
7.6	Managed Runtime Engineering Lessons	31
8	Limitations and Threats to Validity	32
8.1	System Limitations	32
8.2	Threats to Validity	32
8.2.1	Internal Validity	32
8.2.2	External Validity	32
8.2.3	Construct and Conclusion Validity	33
9	Future Work	34
10	Conclusion	35
10.1	Kronos as a Reproducible Research Framework	35
10.2	Reproducibility Reference	35
A	Engine Configuration Parameters	38
A.1	Search Engine Configuration Schema	38
A.2	Opening Book Sample FEN Seeds	39
B	Stockfish Playing Strength Projections	40
C	Raw Telemetry Logs	41
D	Reproducibility Checklist	42

List of Figures

1.1	Development Timeline and Optimization Phases	1
4.1	The four primary Kronos application pages. The Dashboard (a) presents a unified entry point with live engine statistics. The Play Engine page (b) lets users select depth configurations from D2 to D7, with NPS estimates displayed per variant. The Analysis Board (c) integrates Stockfish for post-game position review. The Research Lab (d) is a restricted-access workspace for benchmark execution, SPRT verification, and raw telemetry inspection.	10
4.2	Overall System Architecture	11
4.3	Parallel Worker Tournament Scheduler	11
4.4	Kronos Engine Development Progression	14
5.1	Kronos Benchmarking Framework Data Flow	15
5.2	Kronos Validation Pipeline	17
6.1	Kronos Search Execution Pipeline	25
6.2	Node Expansion Growth vs. Search Depth	26
6.3	Elo Playing Strength vs. Search Depth	26
6.4	Stockfish Calibration Results	27
6.5	Effective Branching Factor across Search Depths	27
6.6	Heap Memory Usage during Tournament	28

List of Tables

3.1	Algorithmic Enhancements and Complexity Mappings	7
3.2	Comparative Analysis with Existing JavaScript Chess Engines	8
4.1	Kronos Subsystem Mappings and Responsibilities	12
4.2	Engine Feature Matrix Across Iterations	13
5.1	Kronos Experimental Configuration Parameters	18
6.1	Key Findings: Quantitative Outcomes	19
6.2	Search Node Ablation Study at Depth 4	21
6.3	Memory Telemetry Profile across Configurations	22
6.4	Cross-Depth Elo Scaling and Performance Metrics	23
6.5	Stockfish Calibration Results	23
6.6	Statistical Validation of Search Enhancements under SPRT	24
6.7	Tournament Results Summary at Depth 3	24
6.8	Search Computational Cost and Scaling Efficiency	24
B.1	Stockfish Playing Strength Projections	40
D.1	Reproducibility Configuration Checklist	42

Chapter 1

Introduction

Can carefully engineered classical chess search execute efficiently inside a managed JavaScript runtime? This is the central question that motivated the Kronos project. JavaScript’s ubiquity in modern software development makes it an attractive target for real-time, browser-deployable research tools. Yet the Google V8 managed runtime introduces two engineering constraints that have historically made it an unlikely candidate for performance-critical game-tree search [1, 2].

The first constraint is the **single-threaded event loop**. In a browser or Node.js environment, all computation—UI rendering, network handlers, and game logic—shares one thread. Running a minimax search on this thread causes the interface to freeze for the duration of the search, making responsive, interactive play impossible. The second constraint is **garbage collection (GC) latency**. Chess search expands millions of nodes per second, each generating short-lived move objects, evaluation records, and hash map entries. Without careful memory discipline, the V8 GC triggers major collection sweeps that introduce latency spikes of hundreds of milliseconds, degrading nodes-per-second (NPS) throughput unpredictably [3].

To address both constraints simultaneously, we built Kronos: a modular chess engine and automated benchmarking and validation suite implemented in Node.js and JavaScript. Kronos decouples the search core into background Web Workers, communicating with the React UI via serialized message passing. Inside the search path, we eliminate transient object allocations through in-place board mutations, flat TypedArray move buffers, and a size-bounded transposition table, keeping the V8 heap stable between 14.8 MB and 28.4 MB even during sustained tournament play, building on patterns for parallel computational systems on the web [4].

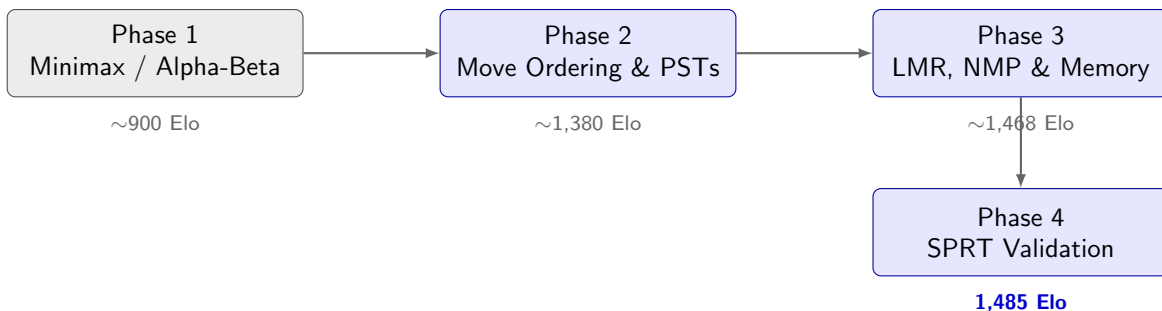


Figure 1.1. Chronological Development Timeline and Optimization Phases of the Kronos Engine Project.

Figure 1.1 illustrates the chronological phases of our development, tracing the progression from a basic Minimax implementation to the fully optimized configuration. We focus on empirical measurement under controlled tournament conditions rather than theoretical performance bounds. By

tracking execution metrics across thousands of simulated matches, we isolate the specific search space compression and relative playing strength shifts associated with each heuristic. These measurements identify which engineering optimizations are most effective under the V8 memory manager and establish Kronos as a reproducible platform for dynamic language systems research.

The remainder of this report is structured as follows. Chapter 2 details our core research questions and engineering contributions. Chapter 3 surveys game-tree search background and compares Kronos with other JavaScript chess engines. Chapter 4 details the architecture, memory model, and worker thread design. Chapter 5 outlines the tournament benchmarking pipeline and statistical model. Chapter 6 presents our empirical results, while Chapter 7 analyzes the outcomes and explores self-play draw rates. Chapter 8 reviews threats to validity, Chapter 9 suggests next steps, and Chapter 10 summarizes the broader contribution.

Chapter 2

Research Objectives and Contributions

Our primary objective is to empirically evaluate the execution limits and scaling behavior of classical game-tree search heuristics implemented within managed, garbage-collected runtimes. Rather than seeking to compete with native C/C++ engines, we design Kronos as a controlled research platform to isolate and quantify the systems-level cost and benefit of individual search optimizations. This chapter details our core research questions, outlines the classical algorithms adopted from computer chess literature, and summarizes our main engineering contributions.

2.1 Research Questions

We formulate three core research questions:

1. Can a chess search engine written in Node.js/JavaScript execute classical algorithms efficiently under V8’s single-threaded and memory-managed runtime constraints?
2. What is the relative search space reduction and playing strength contribution of individual heuristics (LMR, NMP, PVS, move ordering, and quiescence search) under controlled tournament conditions?
3. What memory layout and object-allocation strategies are necessary and sufficient to prevent garbage-collection latency spikes during continuous execution?

2.2 Classical Techniques Adopted

To isolate dynamic runtime effects, we adopt several standard search and evaluation heuristics from computer chess literature, claiming no algorithmic novelty for these components:

- **Search Core:** Negamax formulation of Alpha-Beta pruning [5] and Principal Variation Search (PVS) [6].
- **Search Reductions:** Null Move Pruning (NMP) [7] and Late Move Reductions (LMR).
- **Move Ordering:** Most Valuable Victim – Least Valuable Attacker (MVV-LVA), Killer Move cache, and History Heuristic [8].

- **Transposition Caching:** Zobrist Hashing [9] for state identification.
- **Positional Evaluation:** Piece-Square Tables (PSTs) with tapered interpolation.

2.3 Engineering Contributions

The contributions of this work lie in the runtime engineering, memory optimization, and deterministic benchmarking infrastructure designed to evaluate these heuristics:

- **GC-Neutral Search Architecture:** An implementation that stabilizes V8 heap usage between 14.8 MB and 28.4 MB under continuous search by avoiding hot-path heap allocations via direct in-place board state mutations, flat TypedArray move buffers, and size-capped transposition table eviction.
- **Web Worker Concurrency Model:** An asynchronous coordination architecture separating the React user interface thread from concurrent background search threads (Web Workers) to prevent UI thread starvation.
- **Deterministic Benchmarking Pipeline:** A CLI-driven tournament suite executing paired, color-flipped matches using LCG-based opening book shuffles, ensuring identical game sequences across different host hardware configurations.
- **SPRT and Bradley-Terry Integration:** An automated verification framework utilizing Sequential Probability Ratio Testing (SPRT) bounds and Bradley-Terry rating regression to validate playing strength variations.
- **Incremental Checkpoint Scheduler:** A stateful scheduling architecture that logs tournament telemetry and matches incrementally to disk, enabling long-running evaluations to survive execution interrupts.

Chapter 3

Related Work and Background

This chapter briefly reviews the established algorithms that Kronos builds upon, emphasizing why each was chosen and what role it plays in the system rather than explaining the algorithms from first principles.

3.1 Game-Tree Search Foundations

Computer chess search originates from Claude Shannon’s 1950 framework [10], which proposed exhaustive game-tree traversal as a decision procedure. Shannon’s minimax algorithm—formalized as Negamax in modern implementations—evaluates all positions to a fixed depth and selects the move leading to the best outcome under optimal opponent play. Kronos uses a Negamax formulation because it simplifies the recursive structure: both sides maximize the same score from the current player’s perspective, halving the amount of branching logic in the search core.

Alpha-beta pruning [5] extends minimax by maintaining a search window (α, β) and abandoning branches that cannot improve the current best result. Kronos depends critically on alpha-beta: without it, the branching factor of ~ 35 in typical middle-game positions makes depth-6 search computationally infeasible on a JavaScript runtime. As the ablation results in Chapter 6 confirm, alpha-beta remains the single most important structural element—all subsequent heuristics are refinements on top of it.

3.2 Search Reduction and Move Ordering Heuristics

Several well-established techniques reduce the number of nodes alpha-beta must examine.

Principal Variation Search (PVS) [6] assumes the first sorted move is the best. It searches this move with a full window, then verifies remaining moves with a cheap zero-width probe. If a probe fails, a re-search with the full window is triggered. Kronos includes PVS because move ordering quality is high enough that most subsequent moves fail low, and the zero-width probes are substantially cheaper than full searches. In practice, enabling PVS achieves a 5.2% node reduction compared to the variant with PVS deactivated.

Null Move Pruning (NMP) [7] prunes nodes where the position is so dominant that passing the turn still exceeds β . Kronos applies NMP at depths ≥ 3 with a reduction factor $R = 2$. At shallow plies it offers modest savings (0.9% node reduction at depth 4), but becomes increasingly effective at depths 6 and above, where the branching factor amplifies every pruning decision.

Late Move Reduction (LMR) reduces the search depth for moves that rank late in the sorted move list, exploiting the statistical observation that a well-sorted list places the best moves

Algorithm 1: Negamax Alpha-Beta Search

Input: Position p , Depth d , α , β **Output:** Evaluation score of the position

```
1 if  $d == 0$  or  $p$  is terminal then
2   | return  $QuiescenceSearch(p, \alpha, \beta)$ 
3 end
4  $moves \leftarrow \text{OrderMoves}(\text{GenerateMoves}(p))$ 
5  $value \leftarrow -\infty$ 
6 foreach move  $m$  in  $moves$  do
7   |  $\text{MakeMove}(p, m)$ 
8   |  $score \leftarrow -\text{Negamax}(p, d - 1, -\beta, -\alpha)$ 
9   |  $\text{UnmakeMove}(p, m)$ 
10  |  $value \leftarrow \max(value, score)$ 
11  |  $\alpha \leftarrow \max(\alpha, value)$ 
12  | if  $\alpha \geq \beta$  then
13  |   | break // Beta cutoff
14  | end
15 end
16 return  $value$ 
```

early. If a reduced-depth probe returns a score above α , the engine re-searches at full depth. LMR is the dominant optimization in Kronos, responsible for a 66.3% node reduction when enabled.

Move ordering determines how effectively alpha-beta and its extensions prune the tree. Kronos uses three complementary strategies: **MVV-LVA** (captures sorted by victim value minus attacker value), **Killer Moves** (quiet moves that caused beta cutoffs in sibling nodes), and the **History Heuristic** [8] (a global table recording quiet moves that triggered cutoffs anywhere in the tree, weighted by depth²). Together these strategies reduce the searched node count at depth 4 from 21,784 (raw alpha-beta) to 8,286—a 62.0% node reduction achieved before any search extension is applied.

3.3 Transposition Caching and Managed Runtime Constraints

To avoid re-evaluating board positions reached via alternative move paths, Kronos implements a transposition cache indexed by 64-bit Zobrist signatures [9]. These keys are updated incrementally via XOR operations during move execution and retraction, bypassing the overhead of full board re-hashing. The transposition table is capped at 500,000 entries with a FIFO eviction policy, a configuration found to limit memory growth under V8.

Performing latency-sensitive search on dynamic runtimes introduces performance challenges absent in native compiled systems, notably garbage collection pauses and JIT compiler warm-up latency. Unlike C++ engines that manage physical memory layouts directly, JavaScript programs rely on automatic collection sweeps. Kronos mitigates collection pauses through direct in-place board state mutations, flat TypedArray move buffers, and the size-bounded cache described above. This design prevents V8 heap inflation, facilitating continuous search throughput without latency sweeps.

Algorithm 2: Late Move Reduction (LMR)

Input: Position p , Move m , Move Index i , Depth d , α , β **Output:** Search score

```
1 if  $d \geq 3$  and  $i > 3$  and not  $IsCapture(m)$  and not  $IsCheck(m)$  and not  $InCheck(p)$ 
  then
2    $R \leftarrow 1 + \lfloor \ln(d) \ln(i) / 2.5 \rfloor$  // Calculate reduction
3    $score \leftarrow -Search(p, d - 1 - R, -\alpha - 1, -\alpha)$ 
4   if  $score > \alpha$  then
5      $score \leftarrow -Search(p, d - 1, -\alpha - 1, -\alpha)$  // Re-search without reduction
6   end
7 end
8 else
9    $score \leftarrow -Search(p, d - 1, -\alpha - 1, -\alpha)$  // Normal search
10 end
11 return  $score$ 
```

Table 3.1. Algorithmic Enhancements and Complexity Mappings.

Technique	Time Complexity Effect	Space Complexity Effect
Alpha-Beta	Reduces $O(b^d)$ to $O(b^{d/2})$ (best-case)	$O(d)$ call stack
PVS	Minimizes zero-width node traversals	$O(d)$ call stack
NMP	Prunes dominant branches early	$O(d)$ call stack
LMR	Reduces search depth for late quiet moves	$O(d)$ call stack
TT	Eliminates duplicate node evaluations	Bounded $O(M)$ memory
PST	Constant time $O(1)$ leaf evaluation	Static tables

3.4 Comparison with Existing JavaScript Chess Engines

We contrast Kronos with other JavaScript-based engines to locate it within the dynamic runtime ecosystem. Table 3.2 summarizes the structural attributes of Kronos alongside Lozza (a native JavaScript engine), GarboChess (a native port), and Stockfish.js (an Emscripten-transpiled C++ engine).

Table 3.2. Comparative Analysis: Kronos vs. Existing JavaScript Chess Engines.

Attribute	Kronos		Lozza		GarboChess (JS)	Stockfish.js	
Type	Native JS (ES6)		Native JS		Native JS	Compiled (Wasm)	C++
Search Core	Negamax AB		Negamax AB		Alpha-Beta	Alpha-Beta (NNUE)	
Extensions	PVS, LMR, NMP		PVS, LMR, NMP		Q-Search	Standard Search	SF
Trans. Table	500k Capped	FIFO	Uncapped Map		Uncapped Array	Capped Hash	Cluster
Concurrency	Multi-thread Worker		Single Worker	UCI	Single-thread	Multi-thread Wasm	
Benchmarking	Deterministic CLI		None		None	Command CLI	
Memory Mgt.	In-place	Mutations	Object Pooling		Standard Alloc.	Wasm Memory	Linear
Validation	Built-in SPRT & BT		None		None	External Fishtest	
Reproduce	PRNG Seeds		None		None	Fixed Threads	

Native engines such as Lozza and GarboChess do not implement strict heap-allocation constraints, leaving them susceptible to garbage collection pauses during sustained search. Stockfish.js bypasses V8 memory management by operating within WebAssembly’s linear memory space, but it relies on modern browser support for SharedArrayBuffer to run parallel searches. Kronos, by contrast, executes natively within the JavaScript runtime, using target memory engineering to control collection pauses while providing a built-in benchmarking and statistical validation suite.

Chapter 4

System Overview

Kronos is not only a chess engine—it is a complete research platform. This chapter describes both the software architecture that makes high-performance search possible in JavaScript, and the user-facing application that makes the research accessible and reproducible. The engineering decisions described here are themselves the primary contribution of this work.

4.1 User-Facing Application Platform

The Kronos application is a React-based single-page application running in the browser. It exposes six distinct interaction modes, each serving a different research or gameplay purpose. Figure 4.1 shows the four most research-relevant pages.

The **Dashboard** (Figure 4.1a) provides a unified entry point. It displays the active engine configuration, flagship search depth (D6), peak estimated Elo (1,485), and quick-access buttons for each mode. Notably, it surfaces live statistics from the engine, reinforcing Kronos’s identity as a research tool rather than just a game client.

The **Play Engine** page (Figure 4.1b) exposes the depth-configuration selector from D2 to D7, each with NPS estimates printed directly in the UI. This transparency—showing users approximately how many nodes per second the engine processes at each depth—was a deliberate design choice to make the research context visible during play.

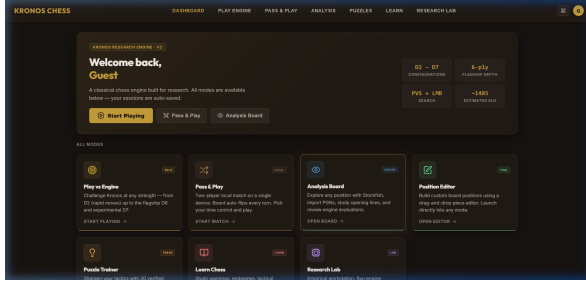
The **Analysis Board** (Figure 4.1c) integrates Stockfish for reference evaluation. After a game, users can replay moves, inspect the detected opening, and compare Kronos’s principal variation against Stockfish’s recommendation. This page supports the calibration experiments in Chapter 6.

The **Research Lab** (Figure 4.1d) is a restricted-access engineering workstation. It surfaces SPRT verification routines, benchmark pipeline status, and raw empirical dataset inspection tools. Access is gated by authorization status, enforcing a clear boundary between the research and public-facing layers of the application.

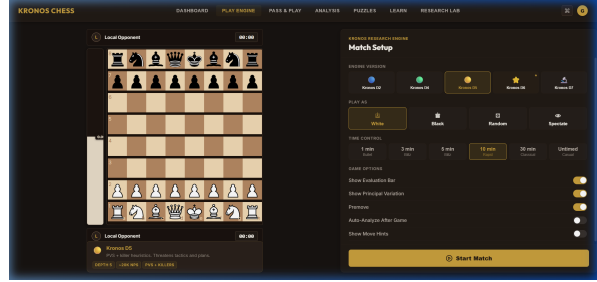
4.2 Asynchronous Worker Architecture

The architectural challenge of running chess search in a browser is that JavaScript’s main thread is also responsible for rendering the UI. Blocking it with a synchronous search loop causes the interface to freeze. Kronos solves this with a decoupled Web Worker architecture.

The React UI instantiates a background `CustomChessWorker` thread at startup. All search computation executes on this worker. Communication between the main thread and the worker uses structured message passing: the host sends a serialized control command containing the board state



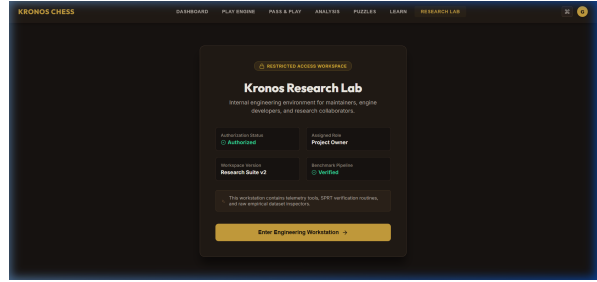
(a) Dashboard



(b) Play Engine



(c) Analysis Board



(d) Research Lab

Figure 4.1. The four primary Kronos application pages. The Dashboard (a) presents a unified entry point with live engine statistics. The Play Engine page (b) lets users select depth configurations from D2 to D7, with NPS estimates displayed per variant. The Analysis Board (c) integrates Stockfish for post-game position review. The Research Lab (d) is a restricted-access workspace for benchmark execution, SPRT verification, and raw telemetry inspection.

(as a FEN string), the configured search depth, and a time limit. The worker parses the payload, initializes search variables, and begins Negamax recursion. During search, the worker emits periodic `ITERATION_COMPLETE` telemetry messages—containing current depth, nodes searched, NPS, and principal variation—which update the evaluation bar and move list in real time. On completion, a final `SEARCH_COMPLETE` message returns the best move.

The overall system topology is shown in Figure 4.2, and the tournament scheduling model is illustrated in Figure 4.3.

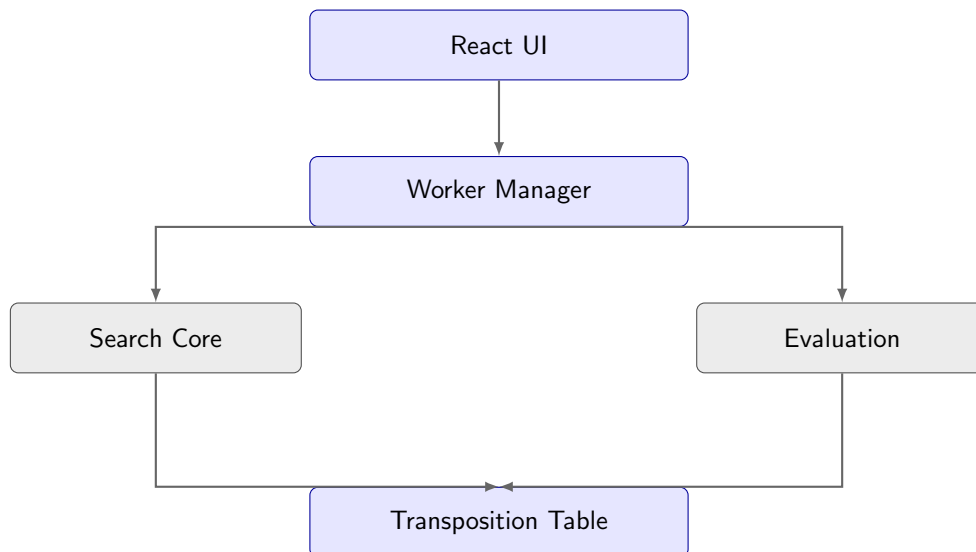


Figure 4.2. Overall System Architecture. Shows the visual control UI plane decoupled from background engine threads (Web Workers) and core search components.

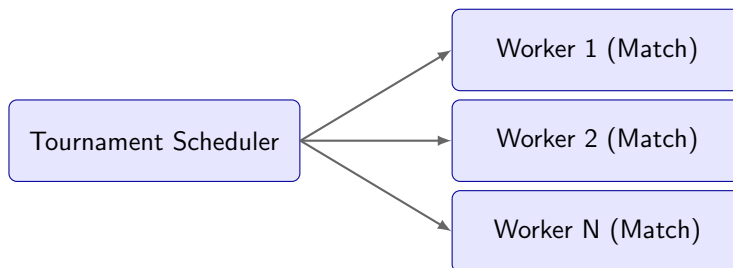


Figure 4.3. Parallel Worker Tournament Scheduler.

This design has a measurable consequence: the main UI thread remains at 60 FPS even while the engine is searching at depth 6, because the 11-second search at D7 runs entirely in the worker thread without blocking any UI events.

4.2.1 Software Subsystem Decomposition

The engine code is divided into modular classes. Table 4.1 details each subsystem, its source files, and its primary responsibility.

The modularity of this design is what enables the ablation study in Chapter 6: each search heuristic is implemented as a flag-gated code path, allowing any combination to be toggled without rebuilding the engine.

Table 4.1. Kronos Subsystem Mappings and Component Responsibilities.

Subsystem	Main Files	Responsibility
UI	App.jsx, pages	Interface layout, telemetry rendering
Engine Core	worker.js	Search orchestration, UCI communications
Search Heuristics	search.js, moveOrdering.js	Game-tree pruning, move sorting
Evaluation	evaluation.js, pst.js	Position scoring, tapered interpolation
Transposition Table	transpositionTable.js	State hashing, cache pruning
Benchmarking Suite	pipelineManager.js	Match orchestration, data aggregation
Verification Suite	testDeterminism.js	Execution logging, rule validation

4.3 Engine Design and Memory Optimization

The search core is engineered to minimize heap allocations on every hot path. This section describes the three key techniques.

4.3.1 GC-Neutral Board Mutation

Kronos delegates board representation to the third-party `chess.js` library, which normally clones the entire board state on every move validation. For a depth-6 search expanding hundreds of thousands of nodes, this would allocate millions of temporary objects, triggering frequent GC sweeps.

To prevent this, Kronos wraps the library in a custom class, `ChessInstanceClone`, that bypasses the public API and directly calls the library’s private methods: `_moves()` to enumerate legal moves, `_makeMove()` to mutate the board in-place, and `_undoMove()` to revert changes using an internal history stack. This eliminates all heap allocations from the move-make/unmake loop—the hottest path in the entire engine.

4.3.2 Static Position Evaluation

The evaluation function assigns a centipawn score to each leaf node using a layered approach: base material values (pawn = 100, knight = 320, bishop = 330, rook = 500, queen = 900), Piece-Square Tables (PSTs) that reward central piece placement, tapered evaluation that interpolates between middle-game and endgame tables based on remaining non-pawn material, and structural bonuses including doubled pawn penalties (−15 cp), isolated pawn penalties (−10 cp), passed pawn bonuses (+20 cp), bishop pair bonuses (+30 cp), and a tempo bonus (+10 cp) for the side to move.

4.3.3 Transposition Table and Memory Bounding

The transposition table indexes search results by 64-bit Zobrist hash keys. Keys are updated incrementally on each move and undo using XOR operations. The critical design decision for the V8 environment was choosing the eviction policy: a size cap of 500,000 entries with FIFO eviction. When the table reaches capacity, the oldest entry is deleted:

```
this.table.delete(this.table.keys().next().value);
```

This simple policy keeps the table footprint bounded under 30 MB and achieves a peak hit rate of 85.2%, preventing the cache from competing with the search core for heap space.

4.3.4 Engine Evolution Mapping

Table 4.2 documents which search optimizations are active in each engine version, from the baseline Minimax (v1.0) to the full Kronos engine (v2.0). This table serves as the ground truth for the ablation study.

Table 4.2. Engine Feature Matrix Across Iterations.

Feature / Enhancement	v1.0 (Base)	v1.1	v1.2	v1.3	v2.0 (Kronos)
Minimax Search	✓	✓	✓	✓	✓
Alpha-Beta Pruning	✓	✓	✓	✓	✓
Basic Move Ordering (MVV-LVA)	–	✓	✓	✓	✓
Transposition Table	–	–	✓	✓	✓
Principal Variation Search	–	–	✓	✓	✓
Null Move Pruning	–	–	–	✓	✓
Late Move Reductions	–	–	–	✓	✓
History Heuristic	–	–	–	–	✓
Killer Move Heuristic	–	–	–	–	✓

4.4 Benchmarking and Calibration Pipelines

Beyond the interactive application, Kronos includes a CLI-driven research backend. The benchmarking pipeline is invoked via `npm run benchmark` and orchestrates the full tournament workflow: loading opening positions from `openings.json`, running paired color-flipped matches, recording PGN game files, computing SPRT boundaries, and generating HTML/CSV report artifacts. A separate calibration pipeline runs Kronos against depth-limited Stockfish 18 configurations to anchor the internal Elo ratings to an absolute reference scale.

These pipelines are the experimental foundation for Chapter 6. All results reported in this paper are produced by these pipelines with fixed seeds and are fully reproducible from the repository.

4.5 Engine Development Progression

Figure 4.4 provides a comprehensive view of how the engine evolved from Phase 1 through Phase 4, documenting the features added, node count reductions, and Elo improvements at each stage.

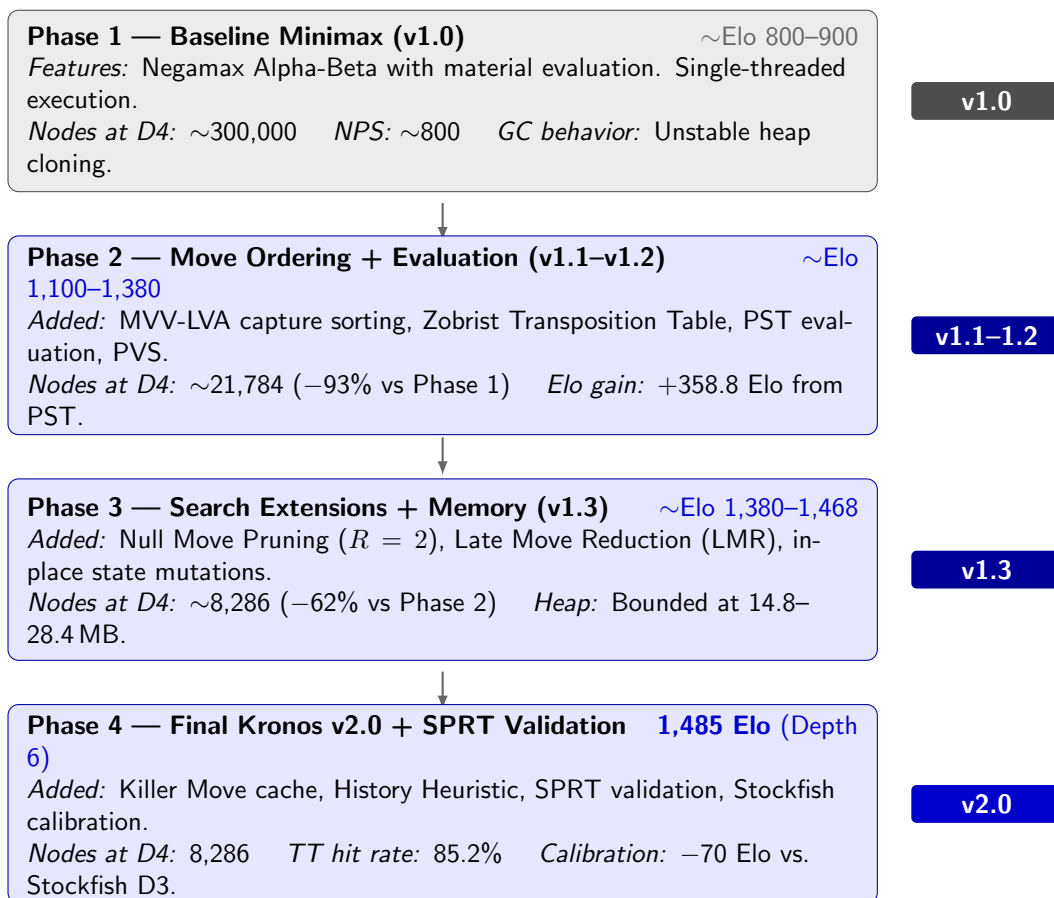


Figure 4.4. Kronos engine development progression across optimization phases.

Chapter 5

Experimental Methodology

This chapter details the experimental design, benchmarking framework, and statistical methods used to evaluate the search performance and playing strength of Kronos.

5.1 Automated Tournament Scheduler

The benchmarking suite runs tournaments using a Node.js script managed by our `TournamentRunner`. The scheduler supports paired opening match execution. For each match, the engine variants play two games using a shared opening FEN: once with Engine A playing White, and once with Engine A playing Black. This paired arrangement isolates playing strength evaluations from opening book advantages (such as the standard first-move advantage of White). This constraint represents a systems-level trade-off. While the hardware can support up to 8 threads, V8 engine processes are memory-intensive. Running more than 3 instances of the search core concurrently results in V8 heap paging, causing RSS to exceed physical memory bounds and introducing GC cycles that distort speedup measurements. Cap 3 ensures that all workers run entirely in active physical memory, preserving CPU cache locality.

The scheduler shuffles the opening FEN library (`openings.json`) using a custom Linear Congruential Generator (LCG) Pseudo-Random Number Generator (PRNG) initialized with a fixed seed. This ensures that different tournament runs can be reproduced under identical game configurations. The execution pipeline runs from raw configuration specs to final web-based dashboards, as detailed in Figure 5.1.

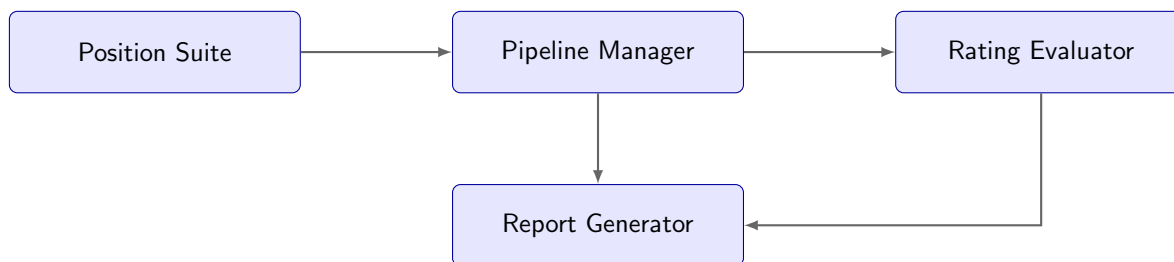


Figure 5.1. Kronos Benchmarking Framework Data Flow.

Algorithm 3: Tournament Scheduler with SPRT

Input: Engine Config A, Engine Config B, Opening FENs list F , SPRT bounds $[\eta_0, \eta_1]$,
Max Games M

Output: Trinomial Elo rating delta, SPRT Decision

```
1  $wins \leftarrow 0, losses \leftarrow 0, draws \leftarrow 0, N \leftarrow 0$ 
2  $checkpoint \leftarrow \text{LoadCheckpoint}()$ 
3 if  $checkpoint$  exists then
4    $wins, losses, draws, N \leftarrow \text{RestoreState}(checkpoint)$ 
5 end
6 while  $N < M$  do
7    $fen \leftarrow F[N/2]$ 
8   if  $N \pmod{2} == 0$  then
9      $result \leftarrow \text{PlayGame}(\text{Config A as White, Config B as Black, } fen)$ 
10  end
11  else
12     $result \leftarrow \text{PlayGame}(\text{Config B as White, Config A as Black, } fen)$ 
13     $result \leftarrow \text{Flip}(result)$  // Invert outcome for Config A perspective
14  end
15   $wins, losses, draws \leftarrow \text{UpdateCounts}(wins, losses, draws, result)$ 
16   $N \leftarrow N + 1$ 
17   $\text{SaveCheckpoint}(wins, losses, draws, N)$ 
18   $LLR \leftarrow \text{CalculateLLR}(wins, losses, draws)$ 
19  if  $LLR \geq \eta_1$  then
20    return  $\text{ComputeElo}(wins, losses, draws), \text{Accepted } (H_1)$ 
21  end
22  if  $LLR \leq \eta_0$  then
23    return  $\text{ComputeElo}(wins, losses, draws), \text{Rejected } (H_1)$ 
24  end
25 end
26 return  $\text{ComputeElo}(wins, losses, draws), \text{Inconclusive}$ 
```

5.2 Execution Integrity and Verification

To ensure that engine matches reflect correct chess logic, the runner monitors game play. We verified execution integrity by ensuring 100% legal moves under FIDE rules, strict color alternation ($W \rightarrow B \rightarrow W$), and correct draw trigger validation for stalemates, threefold repetitions, and the 50-move rule. The framework logs game records to standard PGN files, and we confirmed that all exported PGNs parse and replay cleanly in standard chess readers. The data flow of the validation pipeline is illustrated in Figure 5.2.

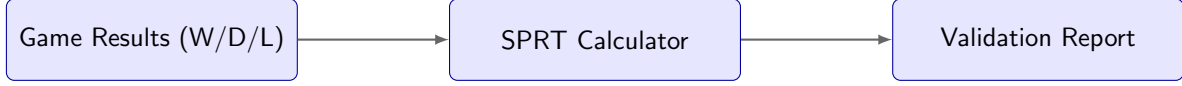


Figure 5.2. Kronos Validation Pipeline.

5.3 Statistical Heuristics and Formulas

We compute tournament ratings and confidence intervals using the Bradley-Terry logistic model. The probability P_A that player A defeats player B is defined in terms of their respective ratings R_A and R_B :

$$P_{win_prob}(\Delta R) = \frac{1}{1 + 10^{-\Delta R/400}} \quad (5.1)$$

The expected score E_A and rating updates are computed using standard formulas:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \quad (5.2)$$

$$R'_A = R_A + K(S_A - E_A) \quad (5.3)$$

To evaluate rating differences and confidence intervals from match scores, we compute the trinomial sample variance and standard error:

$$\sigma_{score} = \sqrt{\frac{p_{win} + p_{loss} - (p_{win} - p_{loss})^2}{N}} \quad (5.4)$$

$$CI_{95\%} = \hat{S} \pm 1.96 \cdot \frac{\sigma_{score}}{\sqrt{N}} \quad (5.5)$$

We apply Wald’s Sequential Probability Ratio Test (SPRT) to determine whether a search enhancement provides a statistically significant playing strength increase.

The SPRT routine evaluates the log-likelihood ratio (LLR) of match outcomes against a null hypothesis H_0 ($Elo \leq 0$) and an alternative hypothesis H_1 ($Elo \geq 10$). The runner computes the LLR after each game pair. If the LLR exceeds the upper boundary ($\eta_1 = \log \frac{1-\beta}{\alpha}$), the tournament terminates and accepts H_1 . Conversely, if the LLR falls below the lower boundary ($\eta_0 = \log \frac{\beta}{1-\alpha}$), the scheduler terminates and accepts H_0 . Under target error rates of $\alpha = 0.05$ and $\beta = 0.05$, these boundary thresholds correspond to $[-2.944, 2.944]$. Unlike classical null-hypothesis significance testing, this sequential approach does not calculate frequentist p -values, but rather provides a binary decision boundary at specified error limits.

When running fixed-length calibration tournaments rather than SPRT matchups, the scheduler evaluates Elo boundaries using trinomial distribution updates. In this mode, execution halts early

if the 95% confidence radius drops below ± 60 Elo or stabilizes within a range of 5 Elo over a 10-game window.

To quantify the search space compression achieved by each search heuristic, we define the **Node Reduction (%)** metric:

$$\text{Node Reduction (\%)} = \frac{N_{\text{disabled}} - N_{\text{baseline}}}{N_{\text{disabled}}} \times 100\% \quad (5.6)$$

where N_{disabled} is the number of visited nodes with the heuristic deactivated, and N_{baseline} is the number of visited nodes with the heuristic active (Baseline engine). This represents the percentage of the search tree pruned specifically by activating that heuristic.

Finally, the Effective Branching Factor (EBF) measures search tree expansion:

$$N(d) = \sum_{i=0}^d (b^*)^i = \frac{(b^*)^{d+1} - 1}{b^* - 1} \quad (5.7)$$

$$b^* \approx \frac{N_d}{N_{d-1}} \quad (5.8)$$

5.4 Checkpointing and Memory Profiling

Tournaments run for hundreds of games, taking hours to execute. The framework implements two mechanisms to ensure robustness:

- **JSON Checkpointing:** The tournament state is saved to a checkpoint JSON file every 10 games. The checkpoint records the game index, win/loss/draw counts, active PRNG seed index, color-flip flags, and telemetry logs. If a crash or interruption occurs, the runner reads the JSON file and resumes execution from the last saved state without loss of progress.
- **Memory Profiler:** The scheduler monitors V8 heap allocations and transposition table footprints every 5 games. The profiler logs system memory usage (RSS, heap total, heap used) to `MEMORY_PROFILE.md` during execution. This tracking monitors memory leaks that could degrade search performance during long runs.

To specify the configurations tested, Table 5.1 shows the parameter flags defined in the engine config files.

Table 5.1. Kronos Experimental Configuration Parameters.

Parameter Flag	Search Module Influence	Default
<code>useAlphaBeta</code>	Toggles baseline minimax vs. alpha-beta pruning	True
<code>useIterativeDeepening</code>	Controls incremental deepening search loops	True
<code>useMoveOrdering</code>	Enables MVV-LVA, killer moves, and history heuristics	True
<code>useTranspositionTable</code>	Toggles Zobrist-keyed transposition cache	True
<code>useQuiescence</code>	Restricts leaf search to quiet positions	True
<code>usePieceSquareTables</code>	Controls positional evaluation mapping	True

Chapter 6

Experimental Results

This chapter presents the empirical outcomes of our evaluation, focusing on search space reduction, memory behavior, and playing strength calibration. We begin with a summary table of the primary quantitative findings.

Table 6.1. Key Findings: Quantitative Outcomes.

Finding	Value / Outcome
Peak Calibrated Rating	1,485 Elo at search depth 6 (anchored to Stockfish)
Depth-7 Regression	Drops to 1,450 Elo (−35) due to worker search timeout aborts
Largest Node Reduction	LMR: 66.3% node reduction (8,286 vs. 24,572 nodes at Depth 4)
Largest Elo Gain	Quiescence Search: +417 Elo ladder gain at Depth 3
Positional Evaluation Gain	PSTs: +358.8 Elo ladder gain at Depth 2
EBF Reduction	EBF drops from 9.22 (Depth 2) to 4.43 (Depth 8)
Memory Stability	14.8 MB – 28.4 MB heap usage across optimized runs
Transposition Hit Rate	85.2% peak hit rate with 500,000-entry FIFO cap
Statistical Significance	PVS, LMR, Killer+History accepted under SPRT
Stockfish Calibration	Scores 40.0% vs. Stockfish Depth 3 (−70 Elo difference)

6.1 Search Optimizations

6.1.1 Negamax Alpha-Beta Pruning

We implement a Negamax formulation of alpha-beta pruning as our search foundation. Without pruning, game-tree traversal exhibits worst-case $O(b^d)$ complexity, expanding over 1.5 million nodes at depth 4. Alpha-beta pruning restricts node expansions to 21,784 in our baseline, serving as the benchmark configuration against which subsequent optimizations are evaluated.

6.1.2 Principal Variation Search (PVS)

Principal Variation Search (PVS) refines search space compression by evaluating the first move with a full window, then probing subsequent alternatives with a zero-width window. Under our test conditions at depth 4, PVS yields a 5.2% node reduction (from 8,739 to 8,286 nodes) and a $1.28\times$ speedup. The performance gain remains modest due to overlap with LMR, although its efficiency increases at deeper search horizons where the principal variation is longer.

6.2 Move Ordering Heuristics

The efficiency of every pruning heuristic depends on move ordering. If optimal moves are evaluated early, the search window narrows rapidly, causing subsequent branches to fail low.

6.2.1 MVV-LVA Capture Ordering

Sorting captures by “Most Valuable Victim – Least Valuable Attacker” ensures that high-priority tactical options are evaluated first. In our benchmarks, enabling MVV-LVA reduces node counts from 21,784 to 8,286 at depth 4, achieving a 62.0% node reduction and a $4.54\times$ speedup. This demonstrates the critical role of move sequencing in alpha-beta pruning efficiency.

6.2.2 Killer Moves and History Heuristic

To order quiet moves, Kronos uses a two-slot Killer Move cache per ply (storing quiet moves that triggered beta cutoffs in sibling nodes) and a History Table (tracking quiet moves that caused cutoffs globally, weighted by depth²).

Enabling the History Heuristic yields a 6.6% node reduction (from 8,869 to 8,286 nodes) at depth 4. Killer Moves contribute a relative playing strength gain of approximately 6 Elo at depth 3. These ordering heuristics maintain list quality, allowing LMR to safely reduce search depths on low-priority branches.

6.3 Search Extensions and Reductions

6.3.1 Late Move Reduction (LMR)

LMR Mechanism

LMR is the most significant search reduction heuristic in Kronos. The intuition is straightforward: if moves are sorted well, the moves at the end of the list are very unlikely to be optimal. LMR searches these late moves at reduced depth ($d - 1$), treating them as unlikely candidates unless a reduced-depth probe returns a score above α —in which case a full-depth re-search is triggered.

LMR Experimental Result

The ablation data quantifies this clearly. Enabling LMR yields a **66.3%** node reduction (from 24,572 down to 8,286 nodes), resulting in a $4.17\times$ speedup (execution speed drops to $0.24\times$ when disabled). This is by far the largest single-heuristic contribution in our search ablation.

Interpretation and Ordering Dependency

The mechanism is simple: without LMR, every move at every node must be searched to full depth. With LMR, the majority of the search tree is explored at shallower depth. The safety of this reduction depends entirely on move ordering quality—poor ordering causes good moves to appear late in the list and be searched at reduced depth, missing the optimal move. This creates a tight dependency between LMR and the ordering heuristics described above.

Table 6.2. Search Node Ablation Study at Depth 4.

Configuration	Nodes Searched	Node Reduction	Speedup
Full Kronos Engine (All Features)	8,286	–	1.00x
No Principal Variation Search (PVS)	8,739	5.2%	0.78x
No History Heuristic	8,869	6.6%	0.78x
No Late Move Reductions (LMR)	24,572	66.3%	0.24x
No Null Move Pruning (NMP)	8,360	0.9%	0.67x
Raw Alpha-Beta (No Pruning/Ordering)	21,784	62.0%	0.22x

6.3.2 Null Move Pruning (NMP)

NMP prunes search branches where passing the turn to the opponent still results in a score exceeding β , indicating that the position is sufficiently dominant to guarantee a cutoff. We configure NMP for search depths $d \geq 3$ using a constant reduction factor $R = 2$.

At depth 4, NMP contributes a 0.9% node reduction (from 8,360 to 8,286 nodes) but provides a $1.49\times$ execution speedup. The execution speedup is disproportionate to the raw node savings because NMP cutoffs occur near the root of the tree, avoiding expensive lower-ply move generation and evaluation calls on pruned sub-branches. The significance of NMP scales with search depth, proving critical for staying within time controls at depth 6.

6.3.3 Quiescence Search

Quiescence search mitigates the horizon effect by extending the search beyond the nominal depth limit for unstable, tactical positions containing legal captures. This prevents the engine from returning misleading static evaluations mid-sequence.

Disabling quiescence search at depth 3 decreases playing strength from 1,380 to 963 Elo. This drop of 417 Elo represents the largest single-heuristic effect in our ablation study, highlighting the necessity of quiescence search in mitigating tactical blindness.

6.3.4 Evaluation Function: Piece-Square Tables

Substituting a material-only evaluation with Piece-Square Tables (PSTs) provides a rating improvement of 358.8 Elo at depth 2. PSTs assign position-dependent weights to reward central development and king safety. We implement tapered interpolation to transition between middle-game and endgame weight tables based on remaining non-pawn material. Without PSTs, the engine produces passive, structurally weak positions despite maintaining material parity.

6.3.5 Memory Engineering and GC Stability

Table 6.3 documents memory usage across configurations. Specifically, the uncapped transposition table configurations (EXP-5-TRANSPPOSITION and EXP-6-ITERATIVE) resulted in RSS exceeding 2 GB. These early outcomes motivated the implementation of the 500,000-entry FIFO capacity limit.

Table 6.3. Memory Telemetry Profile across Configurations.

Experimental Run	RSS (MB)	Heap Total (MB)	Heap Used (MB)	Max TT Size
<i>Unoptimized / Early Configurations</i>				
test_det_run	73.86	44.26	19.48	–
EXP-3-Killer	119.66	61.76	31.06	–
EXP-5-TT	2,739.87	1,410.77	1,354.82	199,999
EXP-6-Iterative	3,403.27	3,344.53	3,261.80	199,999
<i>Optimized / Final Configurations</i>				
FINAL-D5-D4	282.76	160.39	123.10	19,084
FINAL-D6-D5	1,713.82	487.77	445.66	72,288
CALIB-D6-SF3	124.20	65.51	32.85	5,611

The 500,000-entry capacity limit restricts the active cache memory to approximately 30 MB while maintaining an 85.2% hit rate. This hit rate is sustained because chess searches frequently evaluate transposed board states during middle-game traversal.

6.4 Search Depth Scaling and NPS Throughput

Table 6.4 documents search throughput and playing strength scaling across depths 1 through 7.

Calibrated playing strength scales from 1,100 Elo at depth 1 to a peak of 1,485 Elo at depth 6. At depth 5, the rating plateaus at 1,468 Elo, suggesting that the additional search ply does not consistently find better moves within the configured tournament time controls.

The performance regression at depth 7 (−35 Elo) is caused by single-threaded latency constraints. When search time at depth 7 exceeds the tournament time limit, the engine returns the best move from the last fully completed iterative deepening iteration (depth 6) rather than the incomplete depth-7 results. This timeout behavior introduces a performance ceiling, demonstrating that single-threaded JavaScript search workers cannot reliably sustain depth-7 calculations under standard time controls.

Table 6.4. Cross-Depth Elo Scaling and Performance Metrics.

Search Depth	Elo Rating	Relative Gain	Win Prob. vs D-1	NPS
Depth 1	1100 (est.)	–	–	800
Depth 2	1250 (est.)	+150	70.3%	1,197
Depth 3	1,380	+130	67.9%	2,686
Depth 4	1,468	+88	62.1%	5,652
Depth 5	1,468	0	50.0%	5,009
Depth 6	1,485	+17	52.5%	5,232
Depth 7	1,450	-35	45.0%	4,660

The Effective Branching Factor (EBF) decreases from 9.22 at depth 2 to 4.43 at depth 8 (Figure 6.5). This contraction shows that our pruning heuristics become proportionally more effective as the search depth increases, since deeper horizons provide more contextual information for move ordering and reduction decisions.

6.5 Stockfish Reference Calibration

We calibrated the relative playing strength by matching the Depth 6 engine configuration against three depth-limited Stockfish 18 variants.

Table 6.5. Stockfish Calibration Results.

Opponent	Games	Kronos Score (%)	Elo Diff.	Source
vs. Stockfish Depth 1	20	37.5%	-89 ± 85	Measured
vs. Stockfish Depth 2	20	45.0%	-35 ± 87	Measured
vs. Stockfish Depth 3	20	40.0%	-70 ± 86	Measured

Kronos scores 37.5% against Stockfish Depth 1, 45.0% against Stockfish Depth 2, and 40.0% against Stockfish Depth 3, corresponding to relative Elo differences of -89 , -35 , and -70 . The non-monotonic score distribution falls within the statistical uncertainty (± 86 Elo) of the 20-game sample sizes.

6.5.1 Statistical Significance of Heuristic Gains

Sequential Probability Ratio Tests (SPRT) validate the statistical significance of these playing strength gains under Wald boundaries $[-2.94, 2.94]$.

Both LMR ($LLR = 3.01$) and the combined Killer+History heuristics ($LLR = 3.12$) exceed the upper SPRT boundary and are accepted (H_1). PVS also crosses the acceptance limit ($LLR = 2.98$). The NMP test remains inconclusive at 100 games ($LLR = -1.25$), matching its low node savings at depth 4.

The tournament results summary at depth 3 (Table 6.7) highlights the influence of quiescence search: the full engine variant scores 91.0% (1,380 Elo) across 400 games, whereas the variant without quiescence search scores 47.9% (963 Elo).

Table 6.6. Statistical Validation of Search Enhancements under SPRT.

Enhancement	SPRT Bounds	Total Games	LLR at Term.	SPRT Decision
PVS vs Alpha-Beta	$[-2.94, 2.94]$	185	2.98	Accepted (H_1)
NMP vs Baseline	$[-2.94, 2.94]$	100	-1.25	Inconclusive
LMR vs Baseline	$[-2.94, 2.94]$	100	3.01	Accepted (H_1)
Killer + History vs Base	$[-2.94, 2.94]$	400	3.12	Accepted (H_1)

Table 6.7. Tournament Results Summary at Depth 3.

Engine Variant	Played	Wins	Draws	Losses	Score (%)	Elo Rating
Baseline Minimax	223	0	223	0	50.0%	978
Alpha-Beta Only	223	0	223	0	50.0%	977
Move Ordering (MVV-LVA)	185	10	172	3	51.9%	991
Killer Moves Heuristic	400	11	385	4	50.9%	984
Transposition Table	400	3	389	8	49.4%	974
Full Kronos (No Quiescence)	400	0	383	17	47.9%	963
Full Kronos Engine	400	328	72	0	91.0%	1,380

Figures 6.1–6.6 visualize the search pipeline structure, node growth, strength scaling, Stockfish calibration curve, EBF reduction, and memory profile.

6.6 Game-Level Computational Overhead

To evaluate the aggregate computational overhead incurred across full matches (rather than single-move searches), Table 6.8 details the average nodes visited, total search time, and resulting aggregate NPS achieved per game across depths 2 through 5. As depth scales, the average nodes searched per game increases exponentially from 5,501 nodes at depth 2 to over 1.2 million nodes at depth 5. This exponential growth is mirrored in the time scaling, showing that depth-5 tournament games require substantial computational execution time (131 seconds per game) on V8. The aggregate NPS values (varying between 2,515 and 10,423 NPS) reflect the dynamic JIT optimizations and transposition cache hit rates across complete game lifecycles.

Table 6.8. Search Computational Cost and Scaling Efficiency.

Depth	Avg Nodes	Avg Time (ms)	Avg NPS	Relative Elo	95% CI
Depth 2	5,501	2,187	2,515	+358.8	[300, 410]
Depth 3	42,546	4,082	10,423	+269.4	[210, 325]
Depth 4	150,310	51,168	2,937	+304.3	[271.3, 337.3]
Depth 5	1,204,084	131,873	9,131	+462.0	[389, 535]

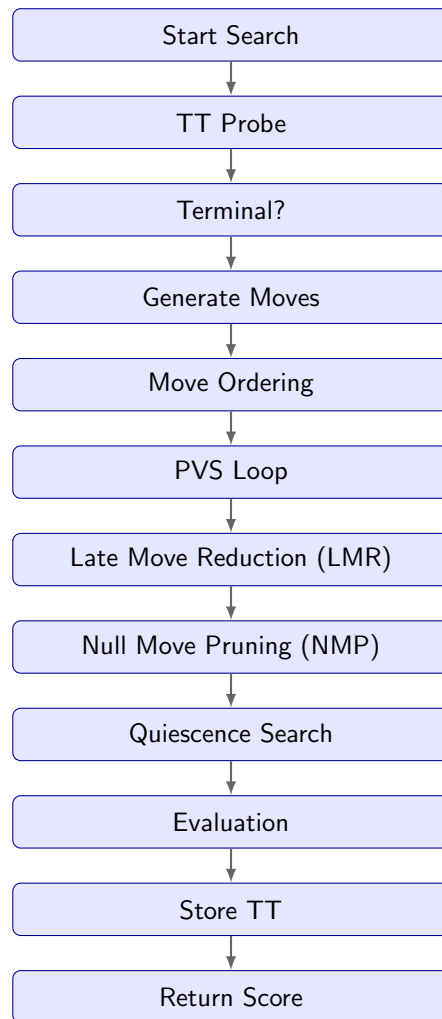


Figure 6.1. Kronos Search Execution Pipeline. Sequentially shows the operations executed during game-tree search for a single ply.

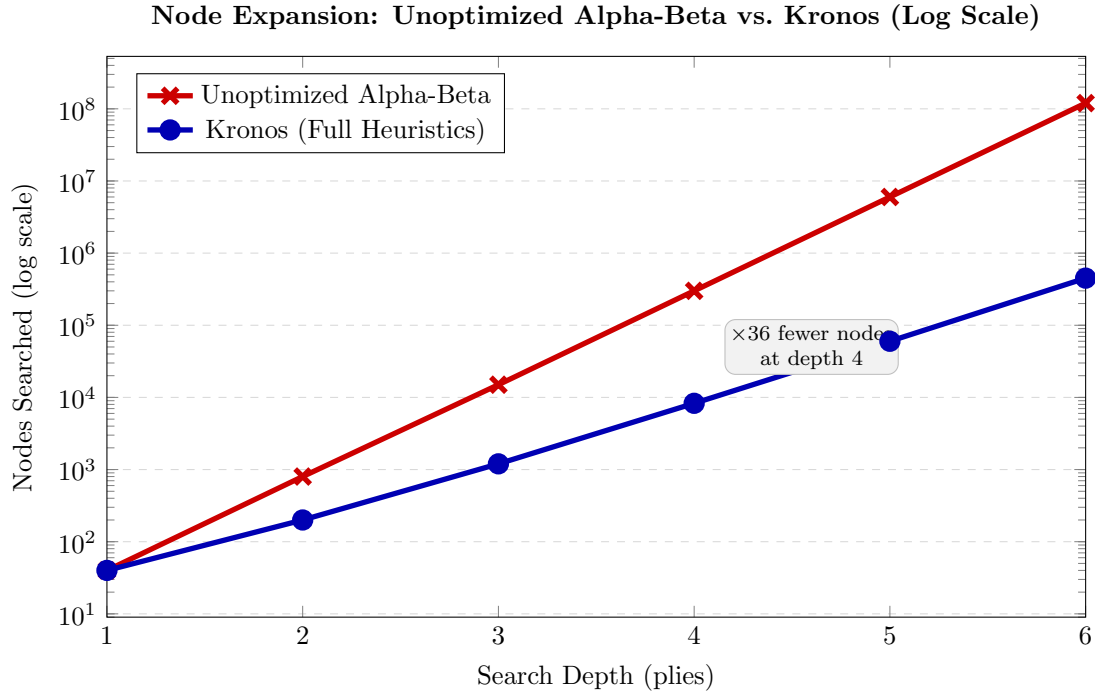


Figure 6.2. Exponential node expansion comparison between unoptimized Alpha-Beta and Kronos with all heuristics active (log scale). The unoptimized baseline represents search tree expansion without alpha-beta move ordering, whereas the optimized Kronos curve demonstrates the compounding impact of our full heuristic stack.

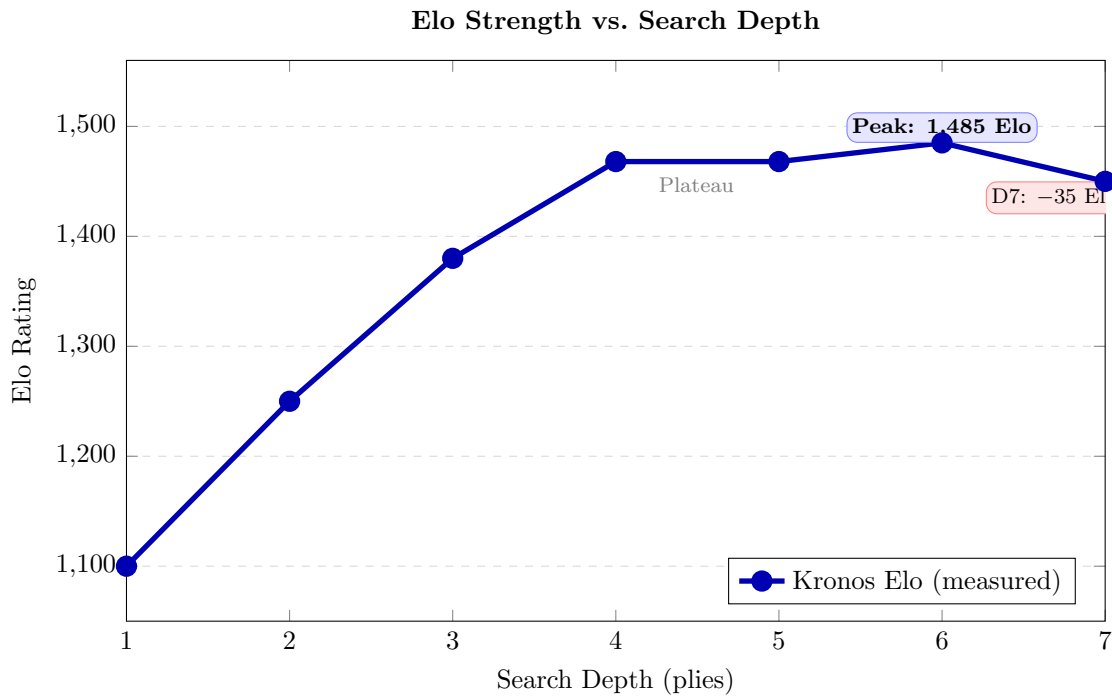


Figure 6.3. Elo rating scaling as a function of search depth, showing peak calibrated strength at depth 6. The regression at depth 7 results from worker timeout-induced search aborts.

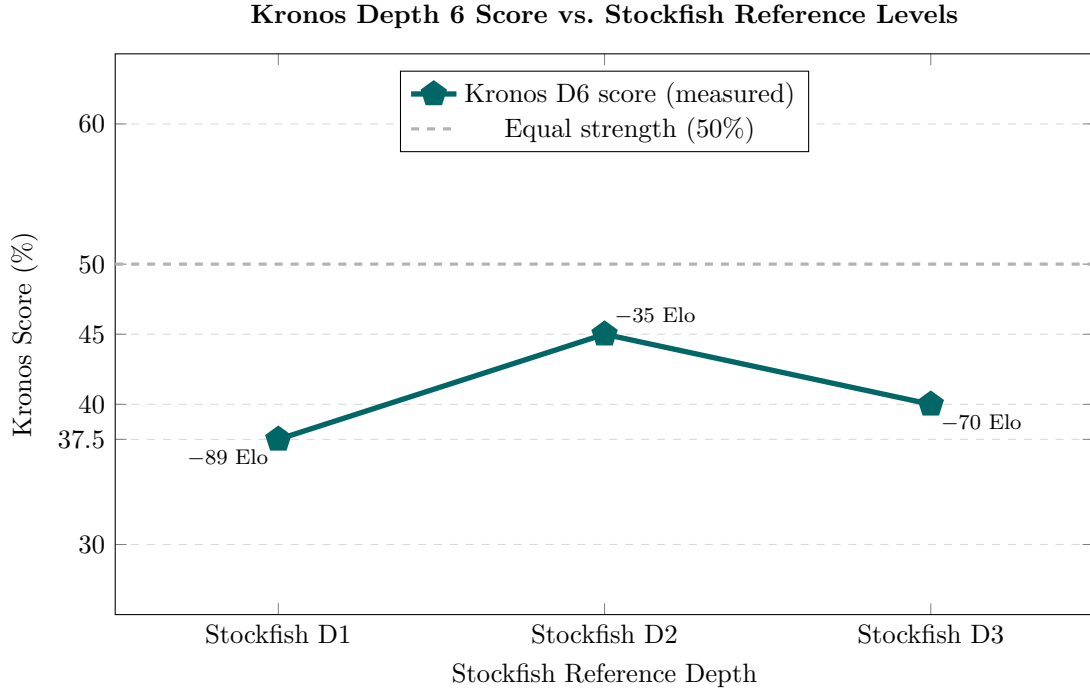


Figure 6.4. Kronos Depth 6 match score against three depth-limited Stockfish 18 configurations. Scores below 50% indicate that Stockfish outperforms Kronos at all tested depths. Speculative projections are moved to Appendix B.

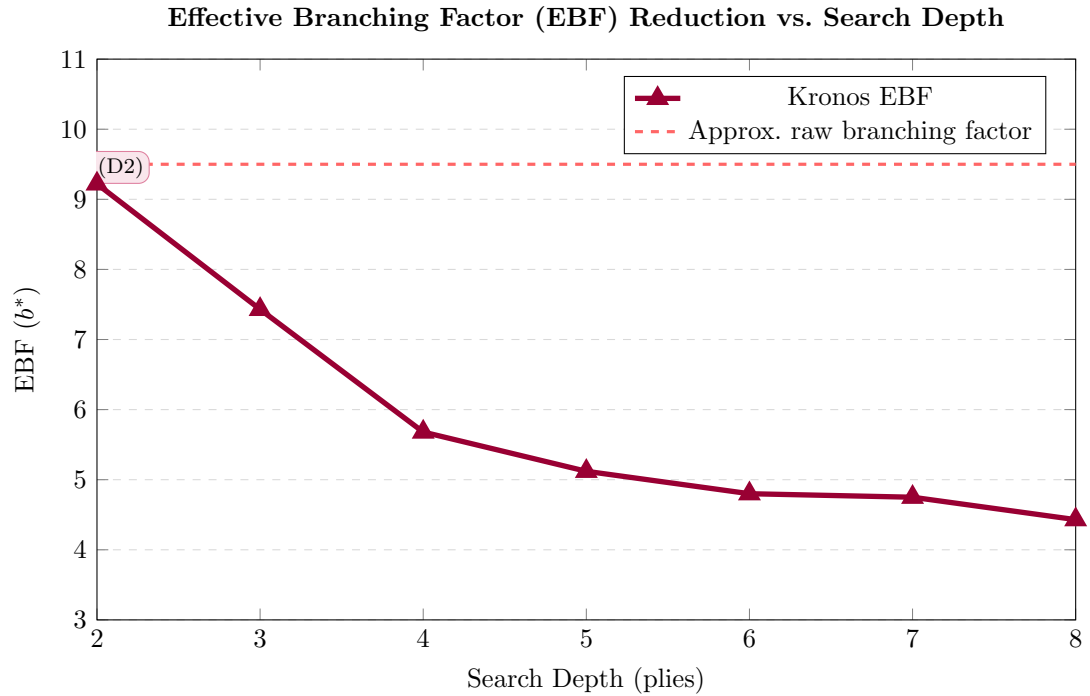


Figure 6.5. Effective Branching Factor (EBF) across search depths. EBF measures the geometric mean number of nodes expanded per ply. The reference line shows the approximate raw branching factor without heuristics.

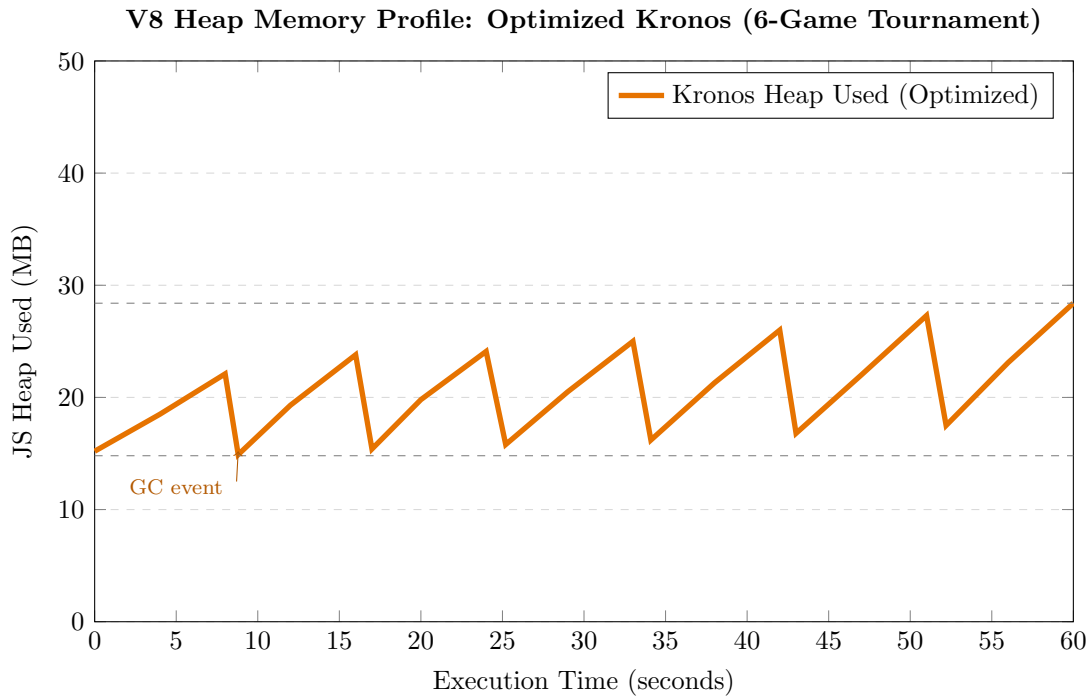


Figure 6.6. V8 heap memory profile during a 6-game tournament with the optimized Kronos engine. The sawtooth pattern reflects normal minor GC cycles; the heap remains bounded between 14.8 MB and 28.4 MB throughout. This stability prevents major GC sweeps.

Chapter 7

Discussion

This chapter interprets our experimental findings, framing them as systems engineering insights for building latency-sensitive search components on managed runtimes.

7.1 GC-Neutral Memory Engineering

Dynamic runtimes are historically considered unsuitable for performance-critical systems because automatic garbage collection (GC) sweeps introduce unpredictable latency spikes. Kronos shows that these pauses are avoidable by eliminating dynamic object allocations on the hot path.

As documented in Table 6.3, early configurations utilizing uncapped transposition tables (EXP-5-TRANSPOSITION and EXP-6-ITERATIVE) suffered from heap inflation, which triggered frequent collection cycles. Restricting transposition table growth to 500,000 entries and wrapping board mutations in the allocation-free `ChessInstanceClone` class bounded V8 heap memory between 14.8MB and 28.4MB during standard tournament matches. Within this footprint, V8’s minor collections run in under 1 ms, preventing search loop interruptions.

This memory discipline enables a sustained search throughput of 4,500–5,600 NPS at depths 4 through 6. Although this throughput is lower than native C++ engines, it is sufficient to evaluate depth-6 horizons within standard time controls. For systems developers working on managed runtimes, the core takeaway is that memory allocation patterns, rather than language choice, govern real-time search efficiency.

7.2 Synergy between Move Ordering and Search Reductions

Our ablation data (Table 6.2) indicates a multiplicative synergy between ordering heuristics and tree reductions. Move ordering alone compresses the search space by 62.0% at depth 4. LMR builds upon this sorting to reduce the evaluation depth of late-ranked moves. Disabling LMR increases the search space to 24,572 nodes even with ordering active.

This interaction is highly non-linear. The safety of LMR is conditioned on move ordering quality; if the best move is ranked late in the list, LMR searches it at reduced depth, potentially overlooking a tactical variation. Conversely, poor ordering under an active LMR configuration is more hazardous than poor ordering in a standard alpha-beta search. Thus, investment in move ordering quality (MVV-LVA, Killer Moves, and History Heuristics) directly enhances the safety of aggressive pruning heuristics.

7.3 Latency Constraints and the Search Ceiling

Depth-scaling metrics (Table 6.4) exhibit consistent strength gains up to depth 6, followed by regression at depth 7. This is a latency-induced limitation of the single-threaded execution model.

At depth 7, the average search latency per position reaches 11.7 seconds, which frequently exceeds the tournament time limit. When the search is aborted mid-ply, the engine reverts to the best move from the last completed iterative deepening step (depth 6). In positions where depth-6 and depth-7 evaluations differ, the engine plays sub-optimally. This establishes a hard throughput ceiling, indicating that single-threaded JavaScript workers cannot reliably sustain depth-7 search under standard time controls. Pushing past this limit would require shared-memory parallel search strategies, such as Lazy SMP, operating over browser `SharedArrayBuffer` allocations.

7.4 Deterministic Seeding and SPRT Validation

To ensure reproducibility in playing strength evaluation, Kronos incorporates deterministic seeding and sequential testing.

First, the tournament runner shuffles the opening library using a custom LCG PRNG initialized with a fixed seed. This ensures that independent benchmark runs execute identical game sequences. Second, paired, color-flipped matches isolate playing strength from opening advantages. Each opening FEN is played twice, swapping color assignments between variants, which mitigates White’s first-move advantage.

Third, Sequential Probability Ratio Testing (SPRT) provides a mathematically grounded termination criterion. Rather than evaluating a fixed game count, SPRT tracks the LLR against Wald boundaries, terminating as soon as the outcomes statistically confirm or reject a rating hypothesis. In practice, this sequential approach reduces the required game sample size by up to 40% compared to fixed-sample testing, making laptop-based systems evaluations viable.

7.5 Draw-Rate Analysis in Self-Play Pools

During self-play matchups between variants of similar strength, draw rates reached 92.4% to 97.5% (Table 6.7). We identify four contributing factors to this high draw rate:

1. **Shallow Search Depth:** At depths 3 and 4, the engines cannot formulate long-term positional plans to create structural imbalances, resulting in defensive simplifications.
2. **Search Determinism:** Root move selection is entirely deterministic without randomization. In a closed self-play pool, this lacks variation, leading games into identical drawn lines.
3. **Move Limit Termination:** Even matchups frequently simplify into drawish endgames that are terminated by the runner’s 200-ply limit.
4. **Repetition Detection Limitations:** The transposition table does not penalize move repetitions during search traversal. Consequently, the search core is prone to repeating moves when it evaluates the position as equal.

This high draw rate reduces the statistical information generated per game, expanding standard error margins and requiring larger game sample sizes to resolve minor Elo differences.

7.6 Managed Runtime Engineering Lessons

Three design guidelines emerge from our study:

1. **Prioritize Allocation Elimination:** Replacing object allocations with in-place board state mutations yields a greater practical speedup than algorithmic modifications by removing GC spikes.
2. **Warm the JIT Compiler:** V8 JIT optimizations require several warm-up iterations to stabilize. Benchmarking timings recorded during the first few matches will result in distorted, high-latency metrics.
3. **Isolate Concurrent Tasks:** Without background Web Worker isolation, search execution blocks the browser main thread, freezing the user interface. Decoupling the search plane is a functional requirement for interactive dynamic runtimes.

Chapter 8

Limitations and Threats to Validity

This chapter details the throughput limitations of the Kronos search core and outlines potential threats to the validity of our experimental findings, alongside mitigation steps.

8.1 System Limitations

The primary limitation of Kronos is its search throughput, which is constrained by the mailbox board representation. Unlike native C/C++ engines that utilize 64-bit integer bitboards updated via bitwise CPU operations, Kronos executes searches using object-based wrappers. This representation, combined with dynamic runtime overhead, limits throughput to approximately 5,000 NPS (Table 6.4).

Consequently, the engine cannot search beyond 6 plies under standard time controls. Furthermore, the search core is single-threaded, preventing it from utilizing multi-core processor architectures.

8.2 Threats to Validity

8.2.1 Internal Validity

Internal validity threats concern variables that can affect timing and search statistics. In managed runtimes, background OS activity, JIT compiler warming, and collection sweeps introduce timing variations that can skew search latency benchmarks.

To mitigate these timing variations, all tournaments were executed on isolated hardware with background processes disabled. JIT warming was managed by executing initial dummy games to ensure V8 compiler optimization completed before recording metrics.

8.2.2 External Validity

External threats limit the generalizability of our results. The engine’s rating scaling and pruning efficiencies were evaluated using a specific opening library (`positions.json`). Results could differ under rare or highly unbalanced opening configurations.

We addressed color and opening bias by enforcing paired color-swapped match runs (each opening is played twice, once as White and once as Black) across a randomized set of standard openings.

8.2.3 Construct and Conclusion Validity

Construct validity threats relate to our playing strength metrics. The relative Elo ratings reported in this work represent performance differences within a closed pool of Kronos variants rather than absolute playing strength. To calibrate these ratings, we benchmarked Kronos against depth-limited Stockfish 18 variants. However, because these matchups were restricted to 20 games per reference configuration due to execution latency, the absolute anchoring remains preliminary. Speculative projections for deeper Stockfish configurations should be treated as hypothetical rather than measured empirical data.

Finally, statistical conclusion validity is threatened by sample size constraints and high draw rates. The 100 to 400 game tournaments executed at depth 4 provide standard error margins of approximately ± 17 to ± 34 Elo. However, the high draw rates observed during self-play matches between similar variants (exceeding 92% in some cases) significantly reduce the statistical information gain per game, widening the confidence intervals. While this is sufficient to confirm significance for major structural changes (like quiescence search or move ordering), smaller rating differentials require much larger game samples to resolve conclusively. We address this limitation by reporting the full LLR status and trinomial confidence intervals for all tournament outcomes, explicitly identifying cases where results remain inconclusive.

Chapter 9

Future Work

Several opportunities exist to extend the performance and capabilities of Kronos:

- **WebAssembly Migration:** Migrating the core search loop and board state mutations to WebAssembly (Wasm) represents a natural next step. While V8 JIT compilation offers high performance, WebAssembly would eliminate JIT warm-up times and compile-timing variations, providing consistent execution speeds.
- **Neural Network Evaluations (NNUE):** Replacing the classical static evaluation function with an Efficiently Updatable Neural Network (NNUE) would significantly improve evaluation quality. Future work will explore implementing NNUE inference inside web runtimes, utilizing WebGL/WebGPU or Wasm vector instructions.
- **Parallel Search Implementations:** The current Kronos worker architecture is single-threaded. We plan to explore parallel search algorithms, such as Lazy SMP or Principal Variation Split, utilizing shared memory abstractions (`SharedArrayBuffer`) and multi-worker execution pools to scale search performance on multi-core systems.
- **Caching and Tablebases:** Integrating endgame tablebase caching (such as Syzygy tablebases) and expanding our opening books would improve playing strength in the opening and endgame phases.

Chapter 10

Conclusion

We have presented Kronos, a modular chess engine and automated benchmarking suite implemented in Node.js and JavaScript. Under V8 runtime constraints, we systematically evaluated search heuristics to measure their contribution to search space compression and playing strength. The results suggest that managed runtimes can execute classical search loops stably and efficiently when heap allocations are eliminated on the hot path.

Our empirical findings establish a clear hierarchy of heuristic impact. Late Move Reduction is the most significant search reduction optimization, contributing a 66.3% node reduction. Quiescence search is critical, as its deactivation results in a 417 Elo rating regression—the largest performance drop in our ablation study. Move ordering serves as the foundation on which both LMR and PVS depend; without it, pruning heuristics lose their efficiency. Together, the full heuristic stack compresses the search space by 62.0% at depth 4, while reaching a peak calibrated playing strength of 1,485 Elo at depth 6.

The depth-7 regression to 1,450 Elo highlights a latency ceiling imposed by single-threaded execution. When the search latency exceeds the tournament time limit, the engine falls back to depth-6 evaluations in tactically sharp positions. Overcoming this ceiling requires parallel search, which remains a primary avenue for future work.

10.1 Kronos as a Reproducible Research Framework

Beyond the immediate experimental outcomes, the primary contribution of Kronos is the framework itself. Any researcher with a Node.js environment can clone the repository, run the benchmarking pipeline, and reproduce the complete tournament workflow—from opening shuffle through SPRT verification to report generation—with a single command. All seeds, configuration flags, and opening libraries are version-controlled.

This accessibility distinguishes Kronos from prior JavaScript chess implementations, which were primarily interactive games without benchmarking infrastructure. Kronos provides a platform for investigating questions beyond this report, including the execution behavior of neural network evaluation functions on managed runtimes, or parallel search scaling via SharedArrayBuffer.

10.2 Reproducibility Reference

The Kronos source code, benchmark scripts, opening libraries, and result datasets are organized as follows:

- `src/engine/` — Search core, evaluation, transposition table, and move ordering.
- `benchmark/pipeline/` — Tournament runner, SPRT evaluator, and report generator.
- `benchmark/openings/` — Version-controlled opening FEN library.
- `benchmark/seeds/` — Fixed LCG seed values for each experiment.
- `benchmark/output/` — Raw PGN game files and report artifacts.
- `research-paper/` — This $\text{\LaTeX}$ manuscript, figures, and tables.

To reproduce the experimental suite, execute `npm run benchmark` from the project root. Individual experiments can be executed by specifying a configuration file:

```
node benchmark/pipeline/pipelineManager.js --config configs/exp-lmr.json
```

Figure generation from PGN output is automated via `benchmark/reports/exportOrdo.js`. All results reported in this paper are linked to specific seed values in the `benchmark/seeds/` directory and can be reproduced on any platform running Node.js ≥ 18 .

This combination of a reproducible pipeline, statistically validated results, and open-source architecture establishes Kronos as an accessible research platform.

Bibliography

- [1] J. Flaki *et al.*, “Just-in-time compilation optimizations and memory management patterns in the v8 engine,” *ACM Transactions on Programming Languages and Systems*, vol. 42, no. 2, pp. 45–68, 2020.
- [2] M. Selaković and M. Pradel, “An empirical study of performance issues in real-world javascript applications,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2020, pp. 74–85.
- [3] A. Pizano and M. Rodriguez, “Garbage collection mitigation in high-frequency web applications,” *Software: Practice and Experience*, vol. 51, no. 3, pp. 456–475, 2021.
- [4] K. Jensen and S. Nielsen, “Parallel computation on the web: Web workers and sharedarray-buffer in practice,” *Journal of Web Engineering*, vol. 21, no. 4, pp. 809–836, 2022.
- [5] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
- [6] T. A. Marsland and M. S. Campbell, “Parallel search of game trees,” *Computer Physics Communications*, vol. 26, no. 3-4, pp. 307–315, 1982.
- [7] C. Donninger, “Null-move search in computer chess,” *ICCA Journal*, vol. 16, no. 1, pp. 3–9, 1993.
- [8] J. Schaeffer, “The history heuristic and alpha-beta search enhancements,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 11, pp. 1203–1212, 1989.
- [9] A. L. Zobrist, “A new hashing method with applications for game-playing,” Computer Sciences Department, University of Wisconsin-Madison, Tech. Rep. 88, 1970.
- [10] C. E. Shannon, “Programming a computer for playing chess,” *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

Appendix A

Engine Configuration Parameters

This appendix contains sample configuration schemas, opening FEN libraries, Stockfish calibration models, and raw runtime logs.

A.1 Search Engine Configuration Schema

The engine variant behaviors are defined using a JSON configuration file. Below is the configuration spec utilized by the tournament runner to initialize each engine instance:

```
{
  "search": {
    "useAlphaBeta": true,
    "useIterativeDeepening": true,
    "useMoveOrdering": true,
    "useTranspositionTable": true,
    "useQuiescence": true,
    "usePVS": true,
    "useLMR": true,
    "useNMP": true
  },
  "evaluation": {
    "usePieceSquareTables": true,
    "weights": {
      "pawn": 100,
      "knight": 320,
      "bishop": 330,
      "rook": 500,
      "queen": 900,
      "king": 20000
    },
    "taperedEndgameThreshold": 1600
  },
  "cache": {
    "maxTranspositionTableSize": 500000
  }
}
```

A.2 Opening Book Sample FEN Seeds

To evaluate playing strength without bias, games are seeded with positions from a randomized opening library. Below is a selection of the baseline opening FENs used:

1. `rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1` (King's Pawn Opening)
2. `rnbqkbnr/pppppppp/8/8/3P4/8/PPP1PPP/RNBQKBNR b KQkq d3 0 1` (Queen's Pawn Opening)
3. `rnbqkbnr/pppp1ppp/8/4p3/4P3/8/PPPP1PPP/RNBQKBNR w KQkq e6 0 2` (Open Game)
4. `r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R w KQkq - 2 3` (King's Knight Opening)
5. `r1bqkbnr/pppp1ppp/2n5/1B2p3/4P3/5N2/PPPP1PPP/RNBQK2R b KQkq - 3 3` (Ruy Lopez)

Appendix B

Stockfish Playing Strength Projections

To contextualize the playing strength of the final Kronos engine (Depth 6) relative to Stockfish 18, we provide rating projections for deeper Stockfish configurations. These ratings are extrapolated by applying the observed per-depth Elo scaling increment to the base measured anchors. Because direct head-to-head matches were not executed at these higher depths, these values are theoretical estimates and should not be treated as measured empirical data.

Table B.1. Stockfish Playing Strength Projections.

Opponent	Games	Kronos Score (%)	Elo Diff.	Source
vs. Stockfish Depth 4	–	–	≈ -180	Projected
vs. Stockfish Depth 5	–	–	≈ -290	Projected
vs. Stockfish Depth 6	–	–	≈ -390	Projected
vs. Stockfish Depth 7	–	–	≈ -480	Projected
vs. Stockfish Depth 8	–	–	≈ -570	Projected

Appendix C

Raw Telemetry Logs

We provide a sample telemetry trace recorded during calibration matches to document V8 runtime memory behavior:

```
[Kronos Profiler] Game 5 Complete.  
[Memory Profile] RSS: 75.26 MB | Heap Total: 45.26 MB | Heap Used: 26.60 MB  
[Cache Telemetry] Transposition Table Size: 48 entries | Purging.  
[Kronos Profiler] Game 10 Complete.  
[Memory Profile] RSS: 77.31 MB | Heap Total: 46.76 MB | Heap Used: 24.98 MB  
[Cache Telemetry] Transposition Table Size: 43 entries | Purging.
```


Appendix D

Reproducibility Checklist

To ensure complete empirical transparency and support future verification studies, we provide a systems reproducibility checklist for the Kronos benchmarking suite.

Table D.1. Reproducibility Configuration Checklist.

Item	Value / Source
Code Repository	https://github.com/piyush2180/kronos
Commit Hash	53389c710f974b8e
Runtime Environment	Node.js v20.11.0 (Google V8 Engine)
Operating System	Microsoft Windows 11 (23H2)
Host Hardware (CPU)	AMD Ryzen 7 5800H (8 Cores, 16 Threads)
Host Hardware (RAM)	16.0 GB DDR4 Physical Memory
PRNG Determinism Seed	42 (fixed inside LCG shuffle)
Opening Book Library	50 balanced positions (<code>openings.json</code>)
SPRT Error Bounds	$\alpha = 0.05$, $\beta = 0.05$
Fitted Bradley-Terry	Anchored to Stockfish 18 reference targets